SAP Subscription Billing Crawler & Automation Engine

End-to-End System Architecture, Auto-POJO Pipeline, Database Persistence & Engineering Manual

Project Version: 1.0.0	**Tech Stack:** Java 21, Maven, PostgreSQL, H2, Jackson, Apache HC5
Platform: Windows / Cross-platform	**Target API:** SAP Subscription Billing Sandbox REST API
Author / Team: Final Year Engineering Project	**Document Type:** Comprehensive Architecture Reference

1. Executive Summary & Problem Statement

Modern enterprise ERP systems like SAP expose dozens of interconnected REST/OData APIs (Bills, Subscriptions, Customers, Products, Rate Plans, Usage Records). However, standard integration workflows suffer from three critical bottlenecks:

- **Rigid Schema Dependencies:** SAP APIs wrap responses in varying keys (bills, subscriptions, value, items, d.results). Traditional systems hardcode DTOs and fail when endpoints evolve.
- **Manual POJO Development:** Creating strongly-typed Java classes for nested enterprise payloads with dozens of fields takes days of boilerplate coding and introduces mapping bugs.
- **Manual Database Schema Maintenance:** Every new service requires writing custom SQL DDL migrations, mapping foreign keys, and maintaining upsert SQL statements.

Our Solution: A 1-Click Automated Intelligence Pipeline. Given just a service name and endpoint URL, the engine auto-fetches live data, auto-infers OpenAPI 3.0 schema, generates strongly-typed Java POJOs via OpenAPI Generator, auto-creates database tables in PostgreSQL (with automatic embedded H2 fallback), and persists live records with zero manual mapping.

2. High-Level System Architecture Diagram

The system is designed with a decoupled 6-tier modular architecture:

3. End-to-End Pipeline Walkthrough

When an engineer or client adds a new service (e.g. subscription), the entire lifecycle occurs in 4 automated stages:

Phase	Component Responsible	Detailed Operational Mechanism
Phase 1: Ingestion & Schema Discovery	SapAutoServiceManager SapApiClient	Fetches live JSON data from the SAP endpoint using Apache HC5 with APIKey header. Evaluates payload: if already OpenAPI spec, stores directly; if live data, dynamically analyzes JSON data types (boolean, integer, float, string, nested objects) and infers a valid OpenAPI 3.0 specification.

Phase 2: Code Generation & Compilation	Maven OpenAPI Plugin SapServiceRegistry	Executes 'openapi-generator-maven-plugin' in non-interactive mode. Produces strongly-typed Java POJOs in target/generated-sources and copies them to 'src/main/java/projectsap/sap//model/'. Registers descriptor dynamically into SapServiceRegistry without static compile-time lock.
Phase 3: Database Synthesis & Ingestion	DatabaseManager (PostgreSQL / H2)	Checks PostgreSQL database 'sap_bills'. Generates dynamic DDL 'CREATE TABLE IF NOT EXISTS s' with primary key 'id', typed relational columns (status, customer_id, market_id, currency, dates), plus 'raw_json TEXT' and timestamps. Upserts live records into the table.
Phase 4: Serving & Visualization	BillServer Frontend (HTML/JS)	Exposes service via REST endpoints: '/api/sync/' (fetch & persist) and '/api/data/' (query stored records). The frontend dashboard displays active services, record counts, and metrics.

4. Core Component Breakdown

1. SapApiClient.java (Network Tier)

Located at: src/main/java/projectsap/sap/bill/client/SapApiClient.java

Implements enterprise Apache HttpClient 5 with RequestConfig for connect timeout (10s) and response timeout (15s). Contains universal fetchFromUrl(String url) adding standard SAP headers (APIKey, Accept: application/json) and validating HTTP 2xx status codes.

2. GenericSapParser.java (Parsing Tier)

Located at: src/main/java/projectsap/sap/common/GenericSapParser.java

Configured with Jackson ObjectMapper, JavaTimeModule, and JsonNullableModule. Tolerates unknown properties and deserializes polymorphic structures. Uses extractArrayNode() to unwrap nested SAP keys (bills, subscriptions, value, items, data, results, d.results). Converts raw JSON to typed List with zero manual field mapping.

3. SapServiceRegistry.java (Registry Tier)

Located at: src/main/java/projectsap/sap/common/SapServiceRegistry.java

Central directory of all recognized SAP microservices. Uses ServiceDescriptor and reflection-safe registerDynamic(serviceId, name, endpoint, pojoClassName, desc) using Class.forName() to avoid hard compile-time circular dependencies.

4. SapAutoServiceManager.java (Automation Tier)

Located at: src/main/java/projectsap/sap/common/SapAutoServiceManager.java

1-Click CLI orchestrator. Reads/fetches endpoints, converts sample JSON tree into OpenAPI 3.0 schema document, executes OpenAPI Generator maven plugin, migrates generated classes, and triggers immediate DB creation & record

persistence.

5. DatabaseManager.java (Data Tier)

Located at: src/main/java/projectsap/sap/bill/db/DatabaseManager.java

Hybrid PostgreSQL (primary on localhost:5432/sap_bills) with embedded H2 fallback. Features ensureServiceTable() for dynamic DDL synthesis, saveServiceData() for ANSI check-then-upserting, and getServiceData() / getServiceCount() for querying.

6. BillServer.java (Presentation & API Tier)

Located at: src/main/java/projectsap/sap/bill/server/BillServer.java

Lightweight, multi-threaded Java HTTP server (port 8080). Handles CORS headers, serves static frontend dashboard, and routes REST endpoints: /api/bills, /api/stats, /api/sync, /api/sync/{service}, /api/data/{service}, /api/services, /api/fetch-log.

7. DatabaseViewer.java & view_db_data.bat (Verification Tier)

Located at: src/main/java/projectsap/sap/bill/db/DatabaseViewer.java

Dedicated CLI inspection utility. Inspects JDBC metadata, counts all records in all user tables, and displays formatted previews directly in the console.

5. Critical Code Snippets Everyone Must Know

The following snippets represent the core intellectual property of this architecture:

Snippet 1: Universal SAP Wrapper Extraction (GenericSapParser.java)

```
public static JsonNode extractArrayNode(JsonNode root) {
    if (root == null) return null;
    if (root.isArray()) return root;

    // Standard SAP REST wrappers
    String[] wrapperKeys = {"bills", "subscriptions", "value", "items", "data", "results", "content"};
    for (String key : wrapperKeys) {
        if (root.has(key) && root.get(key).isArray()) {
            return root.get(key);
        }
    }
    // SAP OData wrappers { "d": { "results": [ ... ] } }
    if (root.has("d")) {
        JsonNode dNode = root.get("d");
        if (dNode.has("results") && dNode.get("results").isArray()) return dNode.get("results");
        if (dNode.isArray()) return dNode;
    }
    return null;
}
```

Snippet 2: Dynamic Table DDL Synthesis (DatabaseManager.java)

```
public static void ensureServiceTable(Connection conn, String tableName, JsonNode sampleItem) throws SQLException {
    if (tableExists(conn, tableName)) return;

    StringBuilder ddl = new StringBuilder("CREATE TABLE IF NOT EXISTS ").append(tableName).append(" (\n");
    ddl.append("    id VARCHAR(255) PRIMARY KEY,\n");

    if (sampleItem != null && sampleItem.isObject()) {
        Iterator<Map.Entry<String, JsonNode>> fields = sampleItem.fields();
        while (fields.hasNext()) {
            Map.Entry<String, JsonNode> entry = fields.next();
            String col = toSnakeCase(entry.getKey());
            JsonNode val = entry.getValue();

            if (val.isObject() && val.has("id")) ddl.append("    ").append(col).append("_id VARCHAR(255),\n");
            else if (val.isBoolean())            ddl.append("    ").append(col).append(" BOOLEAN,\n");
            else if (val.isInt() || val.isLong())ddl.append("    ").append(col).append(" BIGINT,\n");
            else if (val.isFloatingPointNumber())ddl.append("    ").append(col).append(" DECIMAL(18,2),\n");
            else                                 ddl.append("    ").append(col).append(" VARCHAR(255),\n");
        }
    }
    ddl.append("    raw_json TEXT,\n    fetched_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,\n    updated_at TIMESTAMP DEFA
    conn.createStatement().execute(ddl.toString());
}
```

Snippet 3: Dynamic Multi-Service Sync Route (BillServer.java)

```
server.createContext("/api/sync", exchange -> {
    String path = exchange.getRequestURI().getPath();
    String serviceName = path.length() > "/api/sync/".length() ?
                         path.substring("/api/sync/".length()).trim() : "bills";

    if ("bills".equalsIgnoreCase(serviceName)) {
        String json = SapApiClient.fetchBillsFromSAP();
        int count = DatabaseManager.saveBills(json);
        sendJson(exchange, 200, "{\"success\":true,\"count\":" + count + "}");
    } else {
        var desc = SapServiceRegistry.getDescriptor(serviceName);
        String json = SapApiClient.fetchFromUrl(desc.getDefaultEndpoint());
        int count = DatabaseManager.saveServiceData(serviceName, json);
        sendJson(exchange, 200, "{\"success\":true,\"service\":\"" + serviceName + "\",\"count\":" + count + "}");
    }
});
```

6. Practical Walkthrough: Adding a Service from Scratch

Suppose SAP introduces a new microservice: **Customer Management**. Here is the exact end-to-end user workflow:

Step 1: Execute 1-Click Setup Command

The developer runs the setup script with the service name and SAP Sandbox endpoint URL:

```
.\auto_service_setup.bat customer "https://sandbox.api.sap.com/subscriptionbilling/api/customer/v1/customers"
```

Step 2: What Happens Automatically Under the Hood

1. **Live Ingestion:** Connects to SAP, downloads customers payload (JSON).
2. **Schema Generation:** Synthesizes specs/customer-spec.json conforming to OpenAPI 3.0.
3. **POJO Generation:** Produces Customer.java, CustomerAddress.java in projectsap.sap.customer.model.
4. **Registry Update:** customer descriptor registered in SapServiceRegistry.
5. **Database Table Synthesis:** Executes CREATE TABLE IF NOT EXISTS customers (...) in PostgreSQL.
6. **Persistence:** Inserts all customer records into PostgreSQL customers table.

Step 3: Immediate Verification

• **In Terminal:** Run .\view_db_data.bat -> See customers table with row count and columns preview.
• **In Browser:** Open http://localhost:8080/api/data/customers -> Returns stored customer records.
• **Live Sync:** Open http://localhost:8080/api/sync/customer -> Re-crawls SAP and updates PostgreSQL.

7. Database Schema Reference (PostgreSQL / H2)

Table Name	Primary Columns	Key Relational Columns	Special Columns
bills	id VARCHAR(255) PK	document_number BIGINT, billing_type, bill_status, customer_id, market_id, net_amount, gross_amount, due_date, billing_date	raw_json TEXT fetched_at TIMESTAMP updated_at TIMESTAMP
subscriptions	id VARCHAR(255) PK	subscription_id, status, customer_id, market_id, currency, valid_from, valid_until, billing_profile_id	raw_json TEXT fetched_at TIMESTAMP updated_at TIMESTAMP
fetch_log	id SERIAL / IDENTITY PK	bill_count INT, status VARCHAR(50), message TEXT	fetched_at TIMESTAMP
s	id VARCHAR(255) PK	Dynamically inferred scalar fields (_id, status, type, amounts, booleans)	raw_json TEXT fetched_at TIMESTAMP updated_at TIMESTAMP

8. REST API Endpoints Reference

Method	Endpoint	Parameters	Description
GET	/api/bills	status, type, currency	Queries bills from database with optional filters.
GET	/api/bills/{id}	id (Path)	Retrieves single bill record by UUID.
GET	/api/stats	None	Computes revenue, open bill counts, currency metrics, and time-series.
GET / POST	/api/sync	None / service=bills	Crawls SAP Subscription Billing for latest bills and persists to DB.
GET / POST	/api/sync/{service}	service (Path)	Crawls any registered SAP service and persists to its dedicated DB table.
GET	/api/data/{service}	service (Path / Query)	Queries stored records for any service directly from PostgreSQL/H2.
GET	/api/services	None	Returns all registered services, model class, table name, and DB row count.
GET	/api/fetch-log	None	Returns recent crawling activity history and status logs.

9. Operational Cheatsheet (Scripts & Commands)

Script / Command	Purpose	When to Use
run.bat	Compiles code, connects to PostgreSQL, and starts web server on port 8080.	Daily operation, demoing the dashboard, monitoring.
auto_service_setup.bat	1-Click automated POJO generation, registry update, and database table creation.	When integrating any new SAP microservice.
view_db_data.bat	CLI database viewer. Queries PostgreSQL/H2 and prints formatted tables and records.	Quick terminal validation of database records.
test_subscription_pojo.bat	Tests Subscription POJO deserialization and database persistence end-to-end.	Regression testing subscription pipeline.
test_auto_pojo.bat	Tests Bills Auto-POJO conversion and zero-mapping parsing.	Regression testing bills parser.

Engineering Summary: *This project transforms complex enterprise SAP API integration into a fully automated, dynamic, self-healing pipeline. By combining dynamic schema inference, automated code generation, and schema-adaptive SQL persistence, any SAP microservice can be integrated, stored, and visualized in seconds.*